# TED 03 – Gerenciador de Persistência de Dados em Arquivos Estruturados (CSV e JSON)

Relatório Técnico

## 1. Identificação

| | |
|---|---|
| Instituição | UniBalsas – Centro Universitário |
| Curso | Análise e Desenvolvimento de Sistemas |
| Disciplina | Algoritmos e Programação Avançada |
| Atividade | TED 03 – Gerenciador de Persistência de Dados em Arquivos Estruturados (CSV e JSON) |
| Integrantes | Kawê Victor Lima de Oliveira – MA 26.1.16277<br>Luiz Fernando Carvalho da Costa – MA 26.1.13589<br>Joacy Arruda de Assis Junior – MA 25.1.08341 |
| Data | Balsas–MA, 31 de outubro de 2026 |

*Observação: a atividade foi originalmente proposta para realização em dupla; o grupo foi composto por três integrantes, com ciência do professor.*

## 2. Introdução e objetivos

A persistência de dados em arquivos é uma das formas mais elementares e amplamente utilizadas de armazenamento de informações. Os formatos CSV (*Comma-Separated Values*) e JSON (*JavaScript Object Notation*) destacam-se pela simplicidade, legibilidade e interoperabilidade entre sistemas. Entretanto, a manipulação desses arquivos exige cuidados com a codificação de caracteres, com o fechamento correto dos recursos, com falhas de entrada e saída (E/S) e com a preservação dos dados originais em operações que os modificam.

Este trabalho apresenta um módulo em Python para gerenciamento de dados persistidos em arquivos CSV e JSON. Os objetivos específicos foram:

- ler arquivos CSV e JSON e converter os dados entre os dois formatos sem perda de registros ou valores;
- utilizar o gerenciador de contexto `with open(...)` em todas as aberturas de arquivo, com codificação UTF-8;
- tratar as exceções de E/S (`FileNotFoundError`, `PermissionError`, `UnicodeDecodeError`, entre outras) com mensagens claras, sem encerramento inesperado do programa;
- criar automaticamente um backup do arquivo original antes de qualquer operação de escrita ou atualização;
- organizar a solução de forma modular e validá-la por meio de testes com arquivos válidos e situações de erro.



Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

## 3. Descrição da solução

A aplicação foi desenvolvida em Python 3 utilizando exclusivamente a biblioteca padrão (módulos `csv`, `json`, `os` e `datetime`). O código está organizado em um pacote (`gerenciador`) cujos módulos possuem responsabilidades distintas, acessadas por um menu interativo em `main.py`.

### 3.1 Organização do código

| Arquivo | Responsabilidade |
|---|---|
| main.py | Menu interativo: converter CSV para JSON, converter JSON para CSV, visualizar arquivo e adicionar registro. |
| gerenciador/leitura.py | Leitura de CSV (`ler_csv`) e JSON (`ler_json`), retornando listas de dicionários. |
| gerenciador/escrita.py | Escrita de CSV e JSON; chama o backup antes de qualquer gravação. |
| gerenciador/conversao.py | Conversões `csv_para_json` e `json_para_csv`, combinando leitura e escrita. |
| gerenciador/backup.py | Criação do backup com data e hora na pasta `backups/`. |
| gerenciador/erros.py | Tratamento centralizado de exceções e mensagens ao usuário. |
| dados/ | Arquivos de teste válidos e inválidos. |
| testes/test_gerenciador.py | Testes automatizados (`unittest`). |

### 3.2 Manipulação e persistência dos dados

Internamente, todo conjunto de dados é representado como uma lista de dicionários, em que cada dicionário é um registro e as chaves são os nomes dos campos. Essa representação comum é o ponto de ligação entre os formatos: o CSV é lido por `csv.DictReader` e o JSON por `json.load`; na gravação, são usados `csv.DictWriter` e `json.dump`.

Para garantir que a conversão não cause perdas, os valores lidos do CSV são mantidos como texto (por exemplo, `"9.0"` e `"10"` permanecem exatamente como no original). Na conversão de JSON para CSV, o conjunto de colunas é a união ordenada de todas as chaves presentes nos registros; campos ausentes ou nulos geram células vazias, e valores aninhados (listas e objetos) são gravados como texto JSON. A escrita do JSON utiliza `ensure_ascii=False` e indentação, preservando os acentos de forma legível.



Km 5 da BR 230, Fazenda Maliderе IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

## 4. Implementação

### 4.1 Gerenciador de contexto `with open(...)` e codificação UTF-8

Todas as aberturas de arquivo do projeto, inclusive as do backup, utilizam `with open(...)`. Isso assegura o fechamento automático do arquivo ao fim do bloco, mesmo quando uma exceção é levantada. A codificação é fixada em UTF-8 por meio da constante `ENCODING`, e o parâmetro `newline=""` é empregado nos arquivos CSV, conforme recomendação da documentação do módulo `csv`.

```
ENCODING = "utf-8"

def ler_csv(caminho):
    """Lê um CSV e retorna uma lista de dicionários (um por linha)."""
    with open(caminho, "r", encoding=ENCODING, newline="") as arquivo:
        return list(csv.DictReader(arquivo))

def ler_json(caminho):
    """Lê um JSON e retorna uma lista de registros (dicionários)."""
    with open(caminho, "r", encoding=ENCODING) as arquivo:
        dados = json.load(arquivo)
    if isinstance(dados, dict):
        dados = [dados]
    if not isinstance(dados, list) or not all(isinstance(r, dict) for r in dados):
        raise ValueError("o JSON deve conter uma lista de objetos (registros)")
    return dados
```

### 4.2 Leitura, escrita e conversão entre CSV e JSON

A função `ler_json` valida que o conteúdo seja uma lista de objetos (aceitando também um único objeto, que é convertido em lista de um elemento). As funções de escrita realizam o backup antes de abrir o arquivo em modo de escrita. As conversões apenas encadeiam leitura e escrita.

```
def escrever_json(caminho, registros):
    criar_backup(caminho)  # backup ANTES de alterar
    with open(caminho, "w", encoding=ENCODING) as arquivo:
        json.dump(registros, arquivo, ensure_ascii=False, indent=4)
    print(f"[OK] JSON gravado em '{caminho}' ({len(registros)} registros).")

def escrever_csv(caminho, registros):
    criar_backup(caminho)  # backup ANTES de alterar
    campos = _campos(registros)
    with open(caminho, "w", encoding=ENCODING, newline="") as arquivo:
        escritor = csv.DictWriter(arquivo, fieldnames=campos, restval="")
        escritor.writeheader()
        for registro in registros:
            linha = {}
            for chave, valor in registro.items():
                if valor is None:
                    linha[chave] = ""
                elif isinstance(valor, (dict, list)):
                    linha[chave] = json.dumps(valor, ensure_ascii=False)
                else:
                    linha[chave] = valor
            escritor.writerow(linha)
    print(f"[OK] CSV gravado em '{caminho}' ({len(registros)} registros).")

def csv_para_json(caminho_csv, caminho_json):
    registros = ler_csv(caminho_csv)
    escrever_json(caminho_json, registros)
    return registros
```



Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

```
def json_para_csv(caminho_json, caminho_csv):
    registros = ler_json(caminho_json)
    escrever_csv(caminho_csv, registros)
    return registros
```

### 4.3 Tratamento de exceções

O módulo `erros.py` concentra o tratamento das falhas. A função `tratar_erro` identifica o tipo da exceção e exibe uma mensagem objetiva ao usuário, informando o arquivo e a causa. São tratados: arquivo inexistente, falta de permissão, caminho que aponta para uma pasta, erro de codificação, JSON ou CSV inválido, falha de backup, dados inválidos e demais erros de E/S (`OSError`). A função `executar_seguro` envolve qualquer operação em um bloco `try/except`, de modo que o menu continue em execução após um erro.

```
def tratar_erro(erro, caminho=""):
    """Imprime uma mensagem clara de acordo com o tipo de erro."""
    if isinstance(erro, FileNotFoundError):
        msg = f"O arquivo '{caminho}' não foi encontrado. Verifique o caminho informado."
    elif isinstance(erro, PermissionError):
        msg = f"Sem permissão para acessar ou modificar '{caminho}'."
    elif isinstance(erro, IsADirectoryError):
        msg = f"'{caminho}' é uma pasta, não um arquivo."
    elif isinstance(erro, UnicodeError):
        msg = (f"Erro de codificação em '{caminho}'. O arquivo deve estar em UTF-8. "
               f"Detalhe: {erro}")
    elif isinstance(erro, json.JSONDecodeError):
        msg = (f"O arquivo '{caminho}' não é um JSON válido "
               f"(linha {erro.lineno}, coluna {erro.colno}): {erro.msg}")
    elif isinstance(erro, csv.Error):
        msg = f"O arquivo '{caminho}' não é um CSV válido: {erro}"
    elif isinstance(erro, BackupError):
        msg = f"Backup falhou, operação cancelada para proteger os dados: {erro}"
    elif isinstance(erro, ValueError):
        msg = f"Dados inválidos em '{caminho}': {erro}"
    elif isinstance(erro, OSError):
        msg = f"Erro de entrada/saída com '{caminho}': {erro}"
    else:
        msg = f"Erro inesperado: {erro}"
    print(f"[ERRO] {msg}")
    return msg

def executar_seguro(funcao, *args, caminho="", **kwargs):
    """Executa 'funcao' e trata exceções sem encerrar a aplicação.
    Retorna (True, resultado) em caso de sucesso e (False, None) em caso de erro."""
    try:
        return True, funcao(*args, **kwargs)
    except Exception as erro:  # a aplicação não deve encerrar inesperadamente
        tratar_erro(erro, caminho)
        return False, None
```

### 4.4 Backup automático antes da escrita

A função `criar_backup` é chamada no início de `escrever_json` e `escrever_csv`, portanto antes de qualquer alteração do arquivo. Se o arquivo de destino já existe, seu conteúdo é copiado (em modo binário, preservando os bytes originais) para a pasta `backups/`, com nome contendo data, hora e microssegundos, o que evita sobrescrever backups anteriores. O usuário é informado do sucesso da operação. Se o arquivo ainda não existe, não há o que preservar e a mensagem correspondente é exibida. Caso a criação do backup falhe, é levantada a exceção `BackupError` e a escrita é cancelada, evitando a perda dos dados anteriores.



Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

```
def criar_backup(caminho, pasta_backup=None):
    """Copia o arquivo original para a pasta de backups com data/hora.
    Retorna o caminho do backup, ou None se o arquivo ainda não existe."""
    pasta_backup = pasta_backup or PASTA_BACKUP
    if not os.path.exists(caminho):
        print(f"[BACKUP] '{caminho}' ainda não existe; nenhum backup necessário.")
        return None
    try:
        os.makedirs(pasta_backup, exist_ok=True)
        nome, extensao = os.path.splitext(os.path.basename(caminho))
        carimbo = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
        destino = os.path.join(pasta_backup, f"{nome}_{carimbo}{extensao}")
        with open(caminho, "rb") as origem:
            conteudo = origem.read()
        with open(destino, "wb") as copia:
            copia.write(conteudo)
    except OSError as erro:
        raise BackupError(str(erro)) from erro
    print(f"[BACKUP] Backup criado com sucesso: {destino}")
    return destino
```

## 5. Testes e evidências de execução

Os testes foram executados no terminal com o comando `python main.py`, utilizando os arquivos da pasta `dados/`. Para cada cenário são descritos os dados de entrada, o procedimento e o resultado esperado, seguidos da captura de tela correspondente.

### 5.1 Leitura de CSV e conversão para JSON

**Procedimento:** opção 1 do menu, com entrada `dados/alunos.csv` (4 registros contendo acentos, como "João Antônio") e saída `saidas/alunos.json`; em seguida, opção 3 para visualizar o arquivo gerado. **Resultado esperado:** mensagem de conversão concluída com 4 registros e JSON com os mesmos valores e acentuação preservada.



Km 5 da BR 230, Fazenda Maliderе IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

```
===== Gerenciador de Persistência CSV/JSON =====
1 - Converter CSV -> JSON
2 - Converter JSON -> CSV
3 - Visualizar arquivo (CSV ou JSON)
4 - Adicionar registro a um arquivo (com backup automático)
0 - Sair

Escolha uma opção: 1
Arquivo de entrada: dados/alunos.csv
Arquivo de saída: saidas/alunos.json
[BACKUP] 'saidas/alunos.json' ainda não existe; nenhum backup necessário.
[OK] JSON gravado em 'saidas/alunos.json' (4 registros).
[OK] Conversão concluída: 4 registros convertidos.

===== Gerenciador de Persistência CSV/JSON =====
1 - Converter CSV -> JSON
2 - Converter JSON -> CSV
3 - Visualizar arquivo (CSV ou JSON)
4 - Adicionar registro a um arquivo (com backup automático)
0 - Sair

Escolha uma opção: 3
Arquivo: saidas/alunos.json
4 registros em 'saidas/alunos.json':
[{'id': '1',
  'nome': 'João Antônio',
  'curso': 'Sistemas de Informação',
  'nota': '8.5'},
 {'id': '2',
  'nome': 'Maria Conceição',
  'curso': 'Engenharia de Software',
  'nota': '9.0'},
 {'id': '3',
  'nome': 'José Ávila',
  'curso': 'Ciência da Computação',
  'nota': '7.25'},
 {'id': '4',
  'nome': 'Ana Beatriz',
  'curso': 'Sistemas de Informação',
  'nota': '10'}]

===== Gerenciador de Persistência CSV/JSON =====
1 - Converter CSV -> JSON
2 - Converter JSON -> CSV
3 - Visualizar arquivo (CSV ou JSON)
4 - Adicionar registro a um arquivo (com backup automático)
0 - Sair

Escolha uma opção:
```

**Figura 1** – Conversão de alunos.csv para alunos.json e visualização do resultado.

### 5.2 Leitura de JSON e conversão para CSV

**Procedimento:** opção 2 do menu, com entrada `dados/produtos.json` (3 produtos) e saída `saidas/produtos.csv`; em seguida, opção 3 para visualizar o CSV. **Resultado esperado:** CSV com cabeçalho (codigo, descricao, preco, estoque) e 3 linhas, preservando "Pão de Queijo Congelado" e "Açaí Cremoso 1L".



Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br




**Figura 2** – Conversão de produtos.json para produtos.csv e visualização do resultado.

### 5.3 Atualização com backup automático

**Procedimento:** opção 4 do menu sobre `saidas/alunos.json`, adicionando um novo registro. **Resultado esperado:** antes da gravação, a aplicação informa a criação do backup em `backups/`; o arquivo passa a ter 5 registros e a cópia anterior permanece íntegra na pasta de backups.



Km 5 da BR 230, Fazenda Maliderе IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

```
===== Gerenciador de Persistência CSV/JSON =====
1 - Converter CSV -> JSON
2 - Converter JSON -> CSV
3 - Visualizar arquivo (CSV ou JSON)
4 - Adicionar registro a um arquivo (com backup automático)
0 - Sair

Escolha uma opção: 4
Arquivo a atualizar: saidas/alunos.json
  id: 5
  nome: Carlos Eduardo
  curso: Direito
  nota: 6.5
[BACKUP] Backup criado com sucesso: backups\alunos_20261031_190425_091406.json
[OK] JSON gravado em 'saidas/alunos.json' (5 registros).
```

**Figura 3** – Adição de registro ao alunos.json com mensagem de backup criado antes da gravação.




**Figura 4** – Conteúdo da pasta backups/ após a atualização.

### 5.4 Tratamento de erros de arquivo

**Procedimento:** opção 1 do menu informando um arquivo inexistente (`alunos.cvs`, digitado com a extensão incorreta). **Resultado esperado:** mensagem iniciada por [ERRO] informando que o arquivo não foi encontrado, com retorno ao menu sem encerramento do programa. Os demais erros tratados (codificação, JSON inválido e caminho que é uma pasta) foram exercitados pelos testes automatizados da seção 5.5, cujas mensagens aparecem na Figura 6.



Km 5 da BR 230, Fazenda Maliderе IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

```
C:\Users\kawel\Downloads\TED03\TED03>python main.py

===== Gerenciador de Persistência CSV/JSON =====
1 - Converter CSV -> JSON
2 - Converter JSON -> CSV
3 - Visualizar arquivo (CSV ou JSON)
4 - Adicionar registro a um arquivo (com backup automático)
0 - Sair

Escolha uma opção: 1
Arquivo de entrada: alunos.cvs
Arquivo de saída: alunos.json
[ERRO] O arquivo 'alunos.cvs' não foi encontrado. Verifique o caminho informado.

===== Gerenciador de Persistência CSV/JSON =====
1 - Converter CSV -> JSON
2 - Converter JSON -> CSV
3 - Visualizar arquivo (CSV ou JSON)
4 - Adicionar registro a um arquivo (com backup automático)
0 - Sair

Escolha uma opção:
```

**Figura 5** – Mensagem de erro exibida para arquivo inexistente, sem encerrar a aplicação.

### 5.5 Testes automatizados

Foram implementados sete testes com `unittest`, cobrindo a conversão sem perda de dados nos dois sentidos, a criação do backup com o conteúdo anterior preservado e os erros de arquivo inexistente, codificação, JSON inválido e caminho que é uma pasta. Execução: `python -m unittest discover -s testes -v`. Na Figura 6 observa-se a mensagem de erro de codificação (byte 0xe3 inválido em UTF-8), a de JSON inválido e, no teste de pasta no lugar de arquivo, a mensagem de falta de permissão, que é a exceção levantada pelo Windows nesse caso. Os sete testes terminaram com resultado OK.

```
C:\Users\kawel\Downloads\TED03\TED03>python -m unittest discover -s testes -v
test_arquivo_inexistente (test_gerenciador.TestGerenciador.test_arquivo_inexistente) ... [ERRO] O arquivo 'nao_existe.csv' não foi encontrado. Verifique o caminho informado.
ok
test_backup_criado_antes_de_sobrescrever (test_gerenciador.TestGerenciador.test_backup_criado_antes_de_sobrescrever) ... [BACKUP] 'C:\Users\kawel\AppData\Local\Temp\tmp3k_f7zcd\x.json' ain
da não existe; nenhum backup necessário.
[OK] JSON gravado em 'C:\Users\kawel\AppData\Local\Temp\tmp3k_f7zcd\x.json' (1 registros).
[BACKUP] Backup criado com sucesso: C:\Users\kawel\AppData\Local\Temp\tmp3k_f7zcd\bk\x_20261031_191240_057320.json
[OK] JSON gravado em 'C:\Users\kawel\AppData\Local\Temp\tmp3k_f7zcd\x.json' (1 registros).
ok
test_csv_para_json_preserva_dados (test_gerenciador.TestGerenciador.test_csv_para_json_preserva_dados) ... [BACKUP] 'C:\Users\kawel\AppData\Local\Temp\tmps_bn66c8\a.json' ainda não existe;
 nenhum backup necessário.
[OK] JSON gravado em 'C:\Users\kawel\AppData\Local\Temp\tmps_bn66c8\a.json' (4 registros).
ok
test_erro_de_codificacao (test_gerenciador.TestGerenciador.test_erro_de_codificacao) ... [ERRO] Erro de codificação em ''. O arquivo deve estar em UTF-8. Detalhe: 'utf-8' codec can't decod
e byte 0xe3 in position 12: invalid continuation byte
ok
test_json_invalido (test_gerenciador.TestGerenciador.test_json_invalido) ... [ERRO] O arquivo '' não é um JSON válido (linha 1, coluna 3): Expecting property name enclosed in double quotes
ok
test_json_para_csv_preserva_registros (test_gerenciador.TestGerenciador.test_json_para_csv_preserva_registros) ... [BACKUP] 'C:\Users\kawel\AppData\Local\Temp\tmphvvaev8k\p.csv' ainda não
existe; nenhum backup necessário.
[OK] CSV gravado em 'C:\Users\kawel\AppData\Local\Temp\tmphvvaev8k\p.csv' (3 registros).
ok
test_pasta_no_lugar_de_arquivo (test_gerenciador.TestGerenciador.test_pasta_no_lugar_de_arquivo) ... [ERRO] Sem permissão para acessar ou modificar ''.
ok

----------------------------------------------------------------------
Ran 7 tests in 0.025s

OK
```

**Figura 6** – Execução dos testes automatizados (7 testes, resultado OK).



Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

## 6. Conclusão

Os objetivos propostos foram alcançados. A aplicação lê e grava arquivos CSV e JSON, converte os dados nos dois sentidos preservando registros e valores, utiliza `with open(...)` com UTF-8 em todas as operações com arquivos, trata as principais exceções de E/S com mensagens claras e cria backup automático antes de qualquer escrita, cancelando a operação caso o backup não possa ser realizado. A separação em módulos facilitou a manutenção e a criação de testes automatizados.

Entre as dificuldades encontradas, destacam-se: (i) a decisão sobre a tipagem dos valores na conversão, pois o CSV armazena apenas texto, optando-se por mantê-los como texto para evitar perdas (por exemplo, zeros à esquerda e casas decimais); (ii) a definição das colunas do CSV quando os registros JSON possuem campos diferentes ou aninhados; (iii) a reprodução de erros de codificação, que exigiu um arquivo de teste salvo em Latin-1; e (iv) garantir que o backup ocorresse sempre antes da abertura do arquivo para escrita. Como possíveis melhorias futuras, citam-se a inferência opcional de tipos, a limitação do número de backups mantidos e a escrita atômica por arquivo temporário.

## 7. Referências

PYTHON SOFTWARE FOUNDATION. **csv — CSV File Reading and Writing**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/csv.html. Acesso em: 31 out. 2026.

PYTHON SOFTWARE FOUNDATION. **json — JSON encoder and decoder**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/json.html. Acesso em: 31 out. 2026.

PYTHON SOFTWARE FOUNDATION. **The with statement**. The Python Language Reference. Disponível em: https://docs.python.org/3/reference/compound_stmts.html#the-with-statement. Acesso em: 31 out. 2026.

PYTHON SOFTWARE FOUNDATION. **Built-in Exceptions**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/exceptions.html. Acesso em: 31 out. 2026.

PYTHON SOFTWARE FOUNDATION. **unittest — Unit testing framework**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/unittest.html. Acesso em: 31 out. 2026.

SHAFRANOVICH, Y. **Common Format and MIME Type for Comma-Separated Values (CSV) Files**. RFC 4180. IETF, 2005. Disponível em: https://www.rfc-editor.org/rfc/rfc4180.

BRAY, T. (ed.). **The JavaScript Object Notation (JSON) Data Interchange Format**. RFC 8259. IETF, 2017. Disponível em: https://www.rfc-editor.org/rfc/rfc8259.



Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br